

# Report Lab 04: Inheritance and Polymorphism

## 1. Additional requirements of AIMS

```

Media.java  Aims.java  CompactDisc.java  Cart.java  Store.java  Disc.java  Track.java  CompareByCostThenTitle.java
E: > Lab04-main > Lab04-main > AimsProject > src > hust > soict > dsai > aims > media > Book.java > Book > toString()

1  package hust.soict.dsai.aims.media;
2
3  import java.util.ArrayList;
4
5  public class Book extends Media {
6      private ArrayList<String> authors;
7
8      public Book(String title, String category, float cost) {
9          super(title, category, cost);
10         this.authors = new ArrayList<>();
11     }
12
13     public void addAuthor(String author) {
14         if (!authors.contains(author)) {
15             authors.add(author);
16         } else {
17             System.out.println("Author " + author + " already exists in the list.");
18         }
19     }
20
21     public void removeAuthor(String author) {
22         if (authors.contains(author)) {
23             authors.remove(author);
24         } else {
25             System.out.println("Author " + author + " is not in the list.");
26         }
27     }
28
29     public ArrayList<String> getAuthors() {
30         return authors;
31     }
32
33     @Override
34     public String toString() {
35         return "Book - " + getTitle() + " - " + getCategory() + " - Authors: " + String.join(", ", authors) +
36             " - Cost: $" + getCost();
37     }
38     @Override
39     public void displayInfo() {
40         System.out.println("Book - " + getTitle() + " - " + getCategory() + " - Authors: " + String.join(", ", authors)
41             + " - Cost: $" + getCost());
42     }
43 }

```

```
J Media.java J Aims.java J CompactDisc.java X J Cart.java J Store.java J Disc.java J Track.java J CompareByCostThenTitle.java
E: > Lab04-main > Lab04-main > AimsProject > src > hust > soict > dsai > aims > media > J CompactDisc.java > CompactDisc > toString()
1 package hust.soict.dsai.aims.media;
2
3 import java.util.ArrayList;
4
5 public class CompactDisc extends Disc implements Playable {
6     private String artist;
7     private ArrayList<Track> tracks;
8
9     public CompactDisc(String title, String category, float cost, String artist, int length, String director) {
10         super(title, category, cost, length, director);
11         this.artist = artist;
12         this.tracks = new ArrayList<>();
13     }
14
15     public String getArtist() {
16         return artist;
17     }
18
19     public void addTrack(Track track) {
20         if (!tracks.contains(track)) {
21             tracks.add(track);
22             System.out.println("Added: " + track.getTitle());
23         } else {
24             System.out.println("Track " + track.getTitle() + " already exists in the CD.");
25         }
26     }
27
28     public void removeTrack(Track track) {
29         if (tracks.contains(track)) {
30             tracks.remove(track);
31         } else {
32             System.out.println("Track " + track.getTitle() + " is not in the CD.");
33         }
34     }
35
36     public int getTotalLength() {
37         int totallength = 0;
38         for (Track track : tracks) {
39             totallength += track.getLength();
40         }
41         return totallength;
42     }
43 }
```

```

28     public void removeTrack(Track track) {
29         if (tracks.contains(track)) {
30             tracks.remove(track);
31         } else {
32             System.out.println("Track " + track.getTitle() + " is not in the CD.");
33         }
34     }
35
36     public int getTotalLength() {
37         int totallength = 0;
38         for (Track track : tracks) {
39             totallength += track.getLength();
40         }
41         return totallength;
42     }
43
44     @Override
45     public void play() {
46         System.out.println("Playing CompactDisc: " + getTitle());
47         System.out.println("Artist: " + getArtist());
48         System.out.println("CD length: " + getTotalLength() + " mins");
49
50         for (Track track : tracks) {
51             track.play();
52         }
53     }
54     @Override
55     public String toString() {
56         return "CompactDisc - " + getTitle() + " - " + getCategory() + " - Artist: " + getArtist() +
57             " - Director: " + getDirector() + " - Total Length: " + getTotalLength() + " mins - Cost: $"
58             + getCost();
59     }
60     @Override
61     public void displayInfo() {
62         System.out.println("CompactDisc - " + getTitle() + " - " + getCategory() + " - Artist: " + artist
63             + " - Director: " + getDirector() + " - Total Length: " + getTotalLength() + " mins - Cost: $"
64             + getCost());
65         System.out.println("Tracks:");
66         for (Track track : tracks) {
67             System.out.println("\t" + track.toString());
68         }
69     }
70 }

```

## 2. Creating the abstract **Media** class

```
J Media.java x J GarbageCreator.java J ConcatenationInLoops.java J NoGarbage.java
E: > Lab04-main > Lab04-main > AimsProject > src > hust > soict > dsai > aims > media > J Media.java > {} hust.soict.dsai.aims.media

1 package hust.soict.dsai.aims.media;
2
3 public abstract class Media implements Comparable<Media> {
4     private int id;
5     private String title;
6     private String category;
7     private float cost;
8
9     private static int nbMedia = 0;
10
11     public Media(String title, String category, float cost) {
12         this.title = title;
13         this.category = category;
14         this.cost = cost;
15         nbMedia++;
16         this.id = nbMedia;
17     }
18
19     public int getId() {
20         return id;
21     }
22
23     public String getTitle() {
24         return title;
25     }
26
27     public void setTitle(String title) {
28         this.title = title;
29     }
30
31     public String getCategory() {
32         return category;
33     }
34
35     public void setCategory(String category) {
36         this.category = category;
37     }
38
39     public float getCost() {
40         return cost;
41     }
42
43     public void setCost(float cost) {
44         this.cost = cost;
45     }
46
47     public abstract void displayInfo();
```

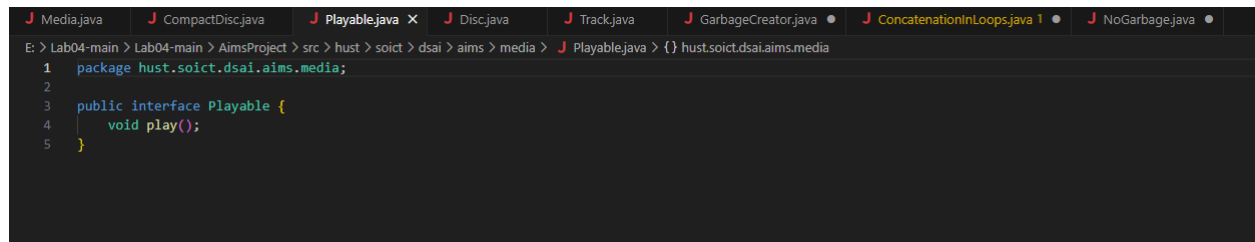
```
J Media.java X J GarbageCreator.java J ConcatenationInLoops.java 1 J NoGarbage.java
E: > Lab04-main > Lab04-main > AimsProject > src > hust > soict > dsai > aims > media > J Media.java > {} hust.soict.dsai.aims.media
3 public abstract class Media implements Comparable<Media> {
31     public String getCategory() {
32         return category;
33     }
34
35     public void setCategory(String category) {
36         this.category = category;
37     }
38
39     public float getCost() {
40         return cost;
41     }
42
43     public void setCost(float cost) {
44         this.cost = cost;
45     }
46
47     public abstract void displayInfo();
48
49     @Override
50     public String toString() {
51         return "Media [Title: " + title + ", Category: " + category + ", Cost: $" + cost + "];";
52     }
53     @Override
54     public int compareTo(Media other) {
55         int titleComparison = this.title.compareToIgnoreCase(other.title);
56         if (titleComparison != 0) {
57             return titleComparison;
58         }
59         return Float.compare(other.cost, this.cost);
60     }
61     @Override
62     public boolean equals(Object obj) {
63         if (obj instanceof Media) {
64             Media other = (Media) obj;
65             return this.title.equalsIgnoreCase(other.getTitle());
66         }
67         return false;
68     }
69 }
```

### 3. Creating the CompactDisc class

```
Media.java CompactDisc.java Disc.java X Track.java GarbageCreator.java ConcatenationInLoops.java NoGarbage.java
E: > Lab04-main > Lab04-main > AimsProject > src > hust > soict > dsai > aims > media > Disc.java > {} hust.soict.dsai.aims.media
1 package hust.soict.dsai.aims.media;
2
3 public class Disc extends Media {
4     private int length;
5     private String director;
6
7     public Disc(String title, String category, float cost, int length, String director) {
8         super(title, category, cost);
9         this.length = length;
10        this.director = director;
11    }
12
13    public int getLength() {
14        return length;
15    }
16
17    public void setLength(int length) {
18        this.length = length;
19    }
20
21    public String getDirector() {
22        return director;
23    }
24
25    public void setDirector(String director) {
26        this.director = director;
27    }
28
29    @Override
30    public void displayInfo() {
31        System.out.println("Disc - " + getTitle() + " - " + getCategory() + " - Director: " + director
32            + " - Length: " + length + " mins - Cost: $" + getCost());
33    }
34 }
```

```
J Media.java    J CompactDisc.java    J Disc.java    J Track.java X    J GarbageCreator.java    J ConcatenationInLoops.java 1    J NoGarbage.java
E: > Lab04-main > Lab04-main > AimsProject > src > hust > soict > dsai > aims > media > J Track.java > {} hust:soict.dsai.aims.media
1  package hust.soict.dsai.aims.media;
2
3  public class Track implements Playable {
4      private String title;
5      private int length;
6
7      public Track(String title, int length) {
8          this.title = title;
9          this.length = length;
10     }
11
12     public String getTitle() {
13         return title;
14     }
15
16     public int getLength() {
17         return length;
18     }
19
20     @Override
21     public void play() {
22         System.out.println("Playing Track: " + title);
23         System.out.println("Track length: " + length + " mins");
24     }
25     @Override
26     public String toString() {
27         return "Track: " + title + " - Length: " + length + " mins";
28     }
29     @Override
30     public boolean equals(Object obj) {
31         if (obj instanceof Track) {
32             Track other = (Track) obj;
33             return this.title.equalsIgnoreCase(other.getTitle()) && this.length == other.getLength();
34         }
35         return false;
36     }
37 }
```

## 4. Create the Playable interface



The screenshot shows an IDE with several tabs at the top: Media.java, CompactDisc.java, Playable.java (active), Disc.java, Track.java, GarbageCreator.java, ConcatenationInLoops.java, and NoGarbage.java. The active tab, Playable.java, contains the following code:

```
1 package hust.soict.dsai.aims.media;
2
3 public interface Playable {
4     void play();
5 }
```

Below the code, the breadcrumb path is displayed: E: > Lab04-main > Lab04-main > AimsProject > src > hust > soict > dsai > aims > media > Playable.java > {} hust.soict.dsai.aims.media

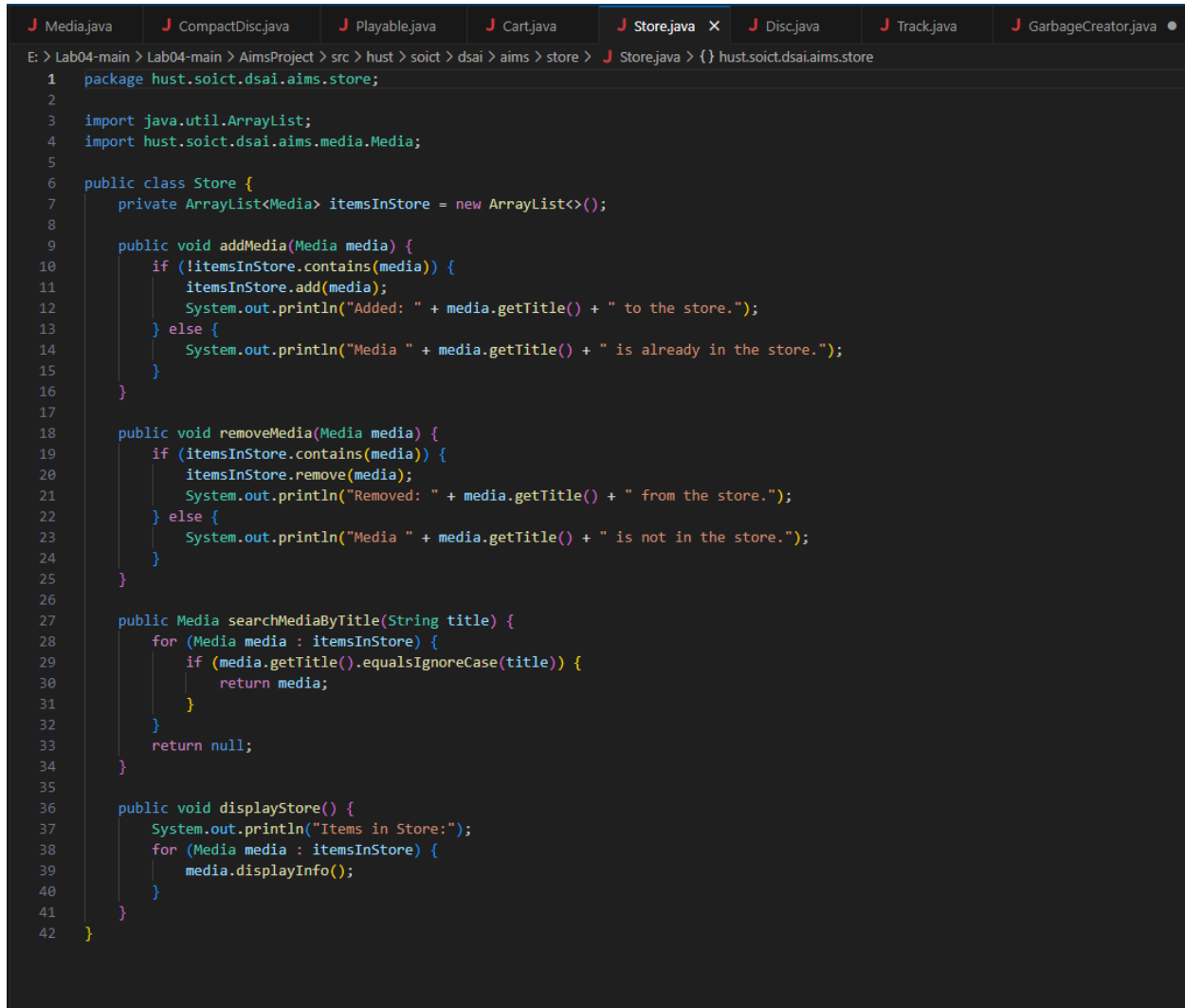
## 5. Update the cart class to work with media

```
J Media.java J CompactDisc.java J Playable.java J Cart.java X J Disc.java J Track.java J GarbageCreator.java •
E: > Lab04-main > Lab04-main > AimsProject > src > hust > soict > dsai > aims > cart > J Cart.java > {} hust.soict.dsai.aims.cart

1 package hust.soict.dsai.aims.cart;
2
3 import java.util.ArrayList;
4 import java.util.Collections;
5
6 import hust.soict.dsai.aims.media.CompareByCostThenTitle;
7 import hust.soict.dsai.aims.media.CompareByTitleThenCost;
8 import hust.soict.dsai.aims.media.Media;
9
10 public class Cart {
11     private ArrayList<Media> itemsOrdered = new ArrayList<>();
12
13     public void addMedia(Media media) {
14         if (!itemsOrdered.contains(media)) {
15             itemsOrdered.add(media);
16             System.out.println("Added: " + media.getTitle());
17         } else {
18             System.out.println("Media " + media.getTitle() + " is already in the cart.");
19         }
20     }
21
22     public void removeMedia(Media media) {
23         if (itemsOrdered.contains(media)) {
24             itemsOrdered.remove(media);
25             System.out.println("Removed: " + media.getTitle());
26         } else {
27             System.out.println("Media " + media.getTitle() + " is not in the cart.");
28         }
29     }
30
31     public float totalCost() {
32         float total = 0;
33         for (Media media : itemsOrdered) {
34             total += media.getCost();
35         }
36         return Math.round(total * 100) / 100.0f;
37     }
38
39     public void displayCart() {
40         System.out.println("Items in Cart:");
41         for (Media media : itemsOrdered) {
42             media.displayInfo();
43         }
44         System.out.println("Total Cost: $" + totalCost());
45     }
46 }
```

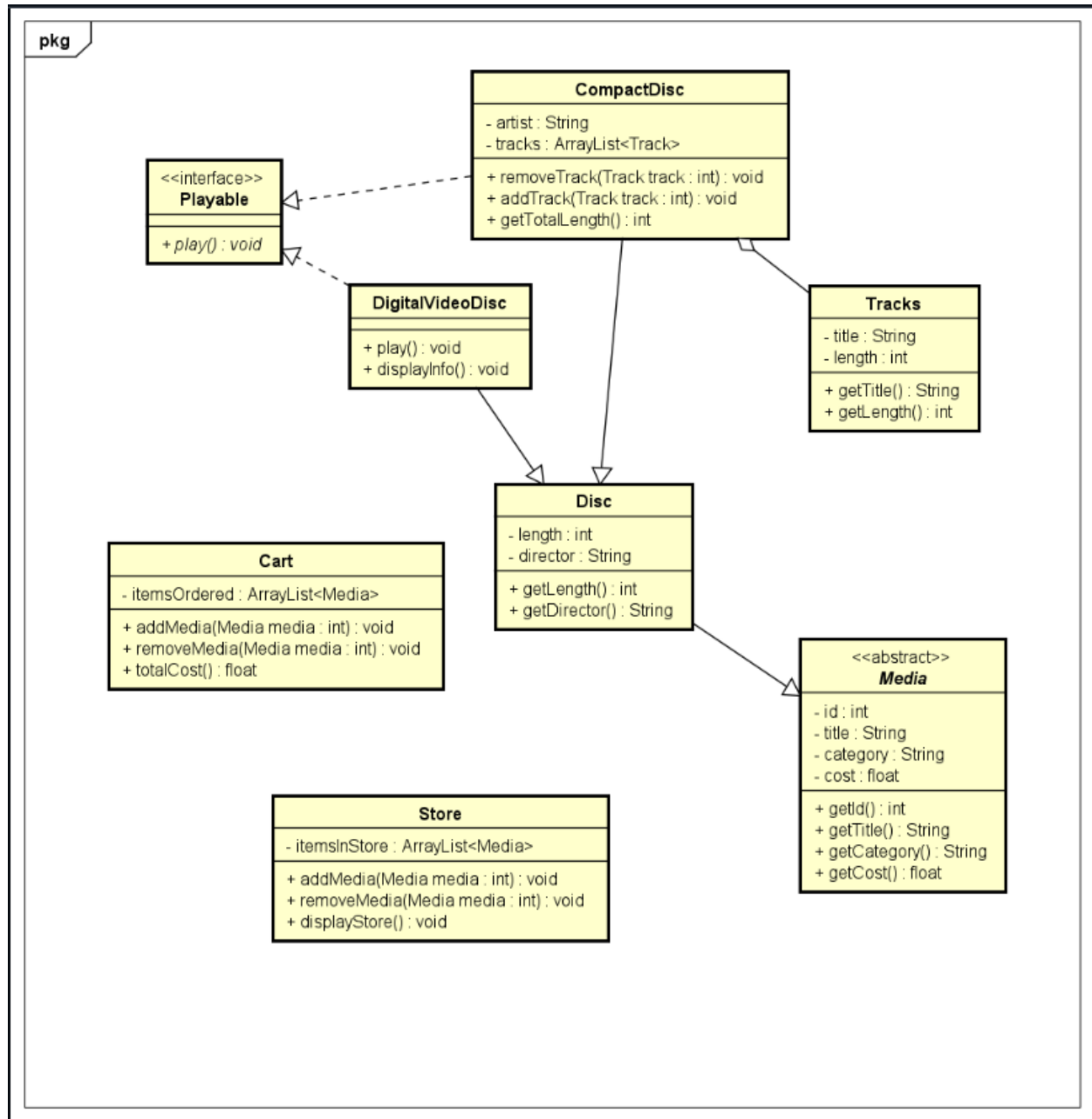
```
J Media.java  J CompactDisc.java  J Playable.java  J Cart.java X  J Disc.java  J Track.java
E: > Lab04-main > Lab04-main > AimsProject > src > hust > soict > dsai > aims > cart > J Cart.java > {} hust.soict.dsai.aims.cart
10  public class Cart {
31      public float totalCost() {
37          }
38
39      public void displayCart() {
40          System.out.println("Items in Cart:");
41          for (Media media : itemsOrdered) {
42              media.displayInfo();
43          }
44          System.out.println("Total Cost: $" + totalCost());
45      }
46
47      public void sortByTitleThenCost() {
48          Collections.sort(itemsOrdered, new CompareByTitleThenCost());
49          System.out.println("\nCart sorted by Title then Cost:");
50          displayCart();
51      }
52
53      public void sortByCostThenTitle() {
54          Collections.sort(itemsOrdered, new CompareByCostThenTitle());
55          System.out.println("\nCart sorted by Cost then Title:");
56          displayCart();
57      }
58
59      public ArrayList<Media> getItemsOrdered() {
60          return itemsOrdered;
61      }
62  }
```

## 6. Update the `store` class to work with `Media`



```
Media.java CompactDisc.java Playable.java Cart.java Store.java X Disc.java Track.java GarbageCreator.java
E: > Lab04-main > Lab04-main > AimsProject > src > hust > soict > dsai > aims > store > Store.java > {} hust.soict.dsai.aims.store
1 package hust.soict.dsai.aims.store;
2
3 import java.util.ArrayList;
4 import hust.soict.dsai.aims.media.Media;
5
6 public class Store {
7     private ArrayList<Media> itemsInStore = new ArrayList<>();
8
9     public void addMedia(Media media) {
10         if (!itemsInStore.contains(media)) {
11             itemsInStore.add(media);
12             System.out.println("Added: " + media.getTitle() + " to the store.");
13         } else {
14             System.out.println("Media " + media.getTitle() + " is already in the store.");
15         }
16     }
17
18     public void removeMedia(Media media) {
19         if (itemsInStore.contains(media)) {
20             itemsInStore.remove(media);
21             System.out.println("Removed: " + media.getTitle() + " from the store.");
22         } else {
23             System.out.println("Media " + media.getTitle() + " is not in the store.");
24         }
25     }
26
27     public Media searchMediaByTitle(String title) {
28         for (Media media : itemsInStore) {
29             if (media.getTitle().equalsIgnoreCase(title)) {
30                 return media;
31             }
32         }
33         return null;
34     }
35
36     public void displayStore() {
37         System.out.println("Items in Store:");
38         for (Media media : itemsInStore) {
39             media.displayInfo();
40         }
41     }
42 }
```

## 7. Constructors of whole classes and parent classes



## 8. Unique item in a list

```
@Override
public boolean equals(Object obj) {
    if (obj instanceof Media) {
        Media other = (Media) obj;
        return this.title.equalsIgnoreCase(other.getTitle());
    }
    return false;
}
```

```
@Override
public boolean equals(Object obj) {
    if (obj instanceof Track) {
        Track other = (Track) obj;
        return this.title.equalsIgnoreCase(other.getTitle()) && this.length == other.getLength();
    }
    return false;
}
```

## 9. Polymorphism with toString() method

```
@Override
public String toString() {
    return "Media [Title: " + title + ", Category: " + category + ", Cost: $" + cost + "];";
}
```

```
@Override
public String toString() {
    return "DVD - " + getTitle() + " - " + getCategory() + " - Director: " + getDirector() +
        " - Length: " + getLength() + " mins - Cost: $" + getCost();
}
```

```
@Override
public String toString() {
    return "Book - " + getTitle() + " - " + getCategory() + " - Authors: " + String.join(", ", authors) +
        " - Cost: $" + getCost();
}
```

```
@Override
public String toString() {
    return "CompactDisc - " + getTitle() + " - " + getCategory() + " - Artist: " + getArtist() +
        " - Director: " + getDirector() + " - Total Length: " + getTotalLength() + " mins - Cost: $"
        + getCost();
}
```

```

@Override
public String toString() {
    return "Track: " + title + " - Length: " + length + " mins";
}

```

## 10. Sort media in the cart

```

@Override
public int compareTo(Media other) {
    int titleComparison = this.title.compareToIgnoreCase(other.title);
    if (titleComparison != 0) {
        return titleComparison;
    }
    return Float.compare(other.cost, this.cost);
}

```

```

@Override
public int compareTo(Media other) {
    if (other instanceof DigitalVideoDisc) {
        DigitalVideoDisc dvd = (DigitalVideoDisc) other;
        int titleComparison = this.getTitle().compareToIgnoreCase(dvd.getTitle());
        if (titleComparison != 0) {
            return titleComparison;
        }
        int lengthComparison = Integer.compare(dvd.getLength(), this.getLength());
        if (lengthComparison != 0) {
            return lengthComparison;
        }
        return Float.compare(dvd.getCost(), this.getCost());
    }
    return 0;
}

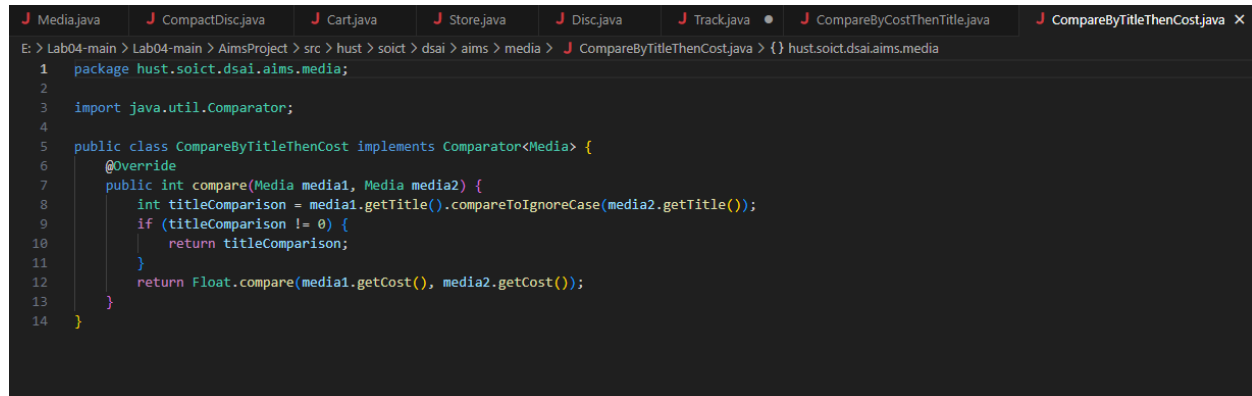
```

Media.java CompactDisc.java Cart.java Store.java Disc.java Track.java CompareByCostThenTitle.java

```

E: > Lab04-main > Lab04-main > AimsProject > src > hust > soict > dsai > aims > media > CompareByCostThenTitle.java > {} hust.soict.dsai.aims.media
1 package hust.soict.dsai.aims.media;
2
3 import java.util.Comparator;
4
5 public class CompareByCostThenTitle implements Comparator<Media> {
6     @Override
7     public int compare(Media media1, Media media2) {
8         int costComparison = Float.compare(media1.getCost(), media2.getCost());
9         if (costComparison != 0) {
10             return costComparison;
11         }
12         return media1.getTitle().compareToIgnoreCase(media2.getTitle());
13     }
14 }

```



```
E: > Lab04-main > Lab04-main > AimsProject > src > hust > soict > dsai > aims > media > CompareByTitleThenCost.java > {} hust.soict.dsai.aims.media
1 package hust.soict.dsai.aims.media;
2
3 import java.util.Comparator;
4
5 public class CompareByTitleThenCost implements Comparator<Media> {
6     @Override
7     public int compare(Media media1, Media media2) {
8         int titleComparison = media1.getTitle().compareToIgnoreCase(media2.getTitle());
9         if (titleComparison != 0) {
10             return titleComparison;
11         }
12         return Float.compare(media1.getCost(), media2.getCost());
13     }
14 }
```

## 11. Create a complete console application in the Aims class

```
J Media.java  J Aims.java X  J CompactDisc.java  J Cart.java  J Store.java  J Disc.java  J Track.java  J CompareBy
E: > Lab04-main > Lab04-main > AimsProject > src > hust > soict > dsai > aims > J Aims.java > {} hust.soict.dsai.aims

1  package hust.soict.dsai.aims;
2
3  import hust.soict.dsai.aims.cart.Cart;
4  import hust.soict.dsai.aims.media.*;
5  import hust.soict.dsai.aims.store.Store;
6
7  import java.util.Collections;
8  import java.util.Scanner;
9
10 public class Aims {
11     public static void main(String[] args) {
12         Scanner scanner = new Scanner(System.in);
13
14         Store store = new Store();
15         Cart cart = new Cart();
16
17         store.addMedia(new Book("Pride and Prejudice", "Romance", 12.50f));
18         store.addMedia(new DigitalVideoDisc("Finding Nemo", "Animation", "Andrew Stanton", 100, 14.99f));
19         store.addMedia(new CompactDisc("Back in Black", "Rock", 18.99f, "AC/DC", 42, "Robert John Lange"));
20         store.addMedia(new Book("Brave New World", "Science Fiction", 13.50f));
21
22         int choice = -1;
23         while (choice != 0) {
24             showMenu();
25             choice = scanner.nextInt();
26             scanner.nextLine();
27             switch (choice) {
28                 case 1:
29                     storeMenu(store, cart, scanner);
30                     break;
31                 case 2:
32                     updateStore(store, scanner);
33                     break;
34                 case 3:
35                     cartMenu(cart, scanner);
36                     break;
37                 case 0:
38                     break;
39                 default:
40                     System.out.println("Invalid option. Please try again.");
41             }
42         }
43     }
44 }
```

```
44
45 public static void showMenu() {
46     System.out.println("AIMS: ");
47     System.out.println("-----");
48     System.out.println("1. View store");
49     System.out.println("2. Update store");
50     System.out.println("3. See current cart");
51     System.out.println("0. Exit");
52     System.out.println("-----");
53     System.out.print("Please choose a number: 0-1-2-3: ");
54 }
55
56 public static void storeMenu(Store store, Cart cart, Scanner scanner) {
57     int choice = -1;
58     while (choice != 0) {
59         System.out.println("Options: ");
60         System.out.println("-----");
61         System.out.println("1. See a media's details");
62         System.out.println("2. Add a media to cart");
63         System.out.println("3. Play a media");
64         System.out.println("4. See current cart");
65         System.out.println("0. Back");
66         System.out.println("-----");
67         System.out.print("Please choose a number: 0-1-2-3-4: ");
68         choice = scanner.nextInt();
69         scanner.nextLine();
70         switch (choice) {
71             case 1:
72                 viewMediaDetails(store, scanner, cart);
73                 break;
74             case 2:
75                 addMediaToCart(store, cart, scanner);
76                 break;
77             case 3:
78                 playMedia(store, scanner);
79                 break;
80             case 4:
81                 cart.displayCart();
82                 break;
83             case 0:
84                 return;
85             default:
86                 System.out.println("Please try again!");
87         }
88     }
89 }
90
```

```
90
91 public static void viewMediaDetails(Store store, Scanner scanner, Cart cart) {
92     System.out.print("Please enter the title: ");
93     String title = scanner.nextLine();
94     Media media = store.searchMediaByTitle(title);
95     if (media != null) {
96         media.displayInfo();
97         mediaDetailsMenu(scanner, media, cart);
98     } else {
99         System.out.println("Not found!");
100     }
101 }
102
103 public static void mediaDetailsMenu(Scanner scanner, Media media, Cart cart) {
104     int choice = -1;
105     while (choice != 0) {
106         System.out.println("Options: ");
107         System.out.println("-----");
108         System.out.println("1. Add to cart");
109         System.out.println("2. Play");
110         System.out.println("0. Back");
111         System.out.println("-----");
112         System.out.print("Please choose a number: 0-1-2: ");
113         choice = scanner.nextInt();
114         scanner.nextLine();
115         switch (choice) {
116             case 1:
117                 addMediaToCartHelper(cart, media);
118                 break;
119             case 2:
120                 if (media instanceof Playable) {
121                     ((Playable) media).play();
122                 } else {
123                     System.out.println("This media cannot be played.");
124                 }
125                 break;
126             case 0:
127                 return;
128             default:
129                 System.out.println("Please try again!");
130         }
131     }
132 }
133
```

```
133
134 public static void addMediaToCartHelper(Cart cart, Media media) {
135     cart.addMedia(media);
136     System.out.println("The media titled \"" + media.getTitle() + "\" has been added to the cart.");
137 }
138
139 public static void addMediaToCart(Store store, Cart cart, Scanner scanner) {
140     System.out.print("Please enter the title of the media you wish to add to the cart: ");
141     String title = scanner.nextLine();
142     Media media = store.searchMediaByTitle(title);
143     if (media != null) {
144         cart.addMedia(media);
145     } else {
146         System.out.println("The specified media could not be found!");
147     }
148 }
149
150 public static void playMedia(Store store, Scanner scanner) {
151     System.out.print("Please enter the title of the media you want to play: ");
152     String title = scanner.nextLine();
153     Media media = store.searchMediaByTitle(title);
154     if (media != null) {
155         if (media instanceof Playable) {
156             ((Playable) media).play();
157         } else {
158             System.out.println("This media is not playable.");
159         }
160     } else {
161         System.out.println("The specified media could not be found!");
162     }
163 }
164
```

```

164
165 public static void updateStore(Store store, Scanner scanner) {
166     System.out.println("Options: ");
167     System.out.println("-----");
168     System.out.println("1. Add a media");
169     System.out.println("2. Remove a media");
170     System.out.println("0. Back");
171     System.out.println("-----");
172     System.out.print("Please choose a number: 0-1-2: ");
173     int choice = scanner.nextInt();
174     scanner.nextLine();
175     switch (choice) {
176         case 1:
177             System.out.print("Enter media type (Book/DVD/CD): ");
178             String mediaType = scanner.nextLine();
179             if (mediaType.equalsIgnoreCase("Book")) {
180                 System.out.print("Enter title, category, cost: ");
181                 String title = scanner.nextLine();
182                 String category = scanner.nextLine();
183                 float cost = scanner.nextFloat();
184                 store.addMedia(new Book(title, category, cost));
185             } else if (mediaType.equalsIgnoreCase("DVD")) {
186                 System.out.print("Enter title, category, director, length, cost: ");
187                 String titleDvd = scanner.nextLine();
188                 String categoryDvd = scanner.nextLine();
189                 String director = scanner.nextLine();
190                 int length = scanner.nextInt();
191                 float costDvd = scanner.nextFloat();
192                 store.addMedia(new DigitalVideoDisc(titleDvd, categoryDvd, director, length, costDvd));
193             } else if (mediaType.equalsIgnoreCase("CD")) {
194                 System.out.print("Enter title, category, artist, director, length, cost: ");
195                 String titleCd = scanner.nextLine();
196                 String categoryCd = scanner.nextLine();
197                 String artist = scanner.nextLine();
198                 String directorCd = scanner.nextLine();
199                 int lengthCd = scanner.nextInt();
200                 float costCd = scanner.nextFloat();
201                 store.addMedia(new CompactDisc(titleCd, categoryCd, costCd, artist, lengthCd, directorCd));
202             } else {
203                 System.out.println("Invalid media type!");
204             }
205             break;

```

```

206         case 2:
207             System.out.print("Enter the title of the media to remove: ");
208             String titleToRemove = scanner.nextLine();
209             Media mediaToRemove = store.searchMediaByTitle(titleToRemove);
210             if (mediaToRemove != null) {
211                 store.removeMedia(mediaToRemove);
212             } else {
213                 System.out.println("Media not found!");
214             }
215             break;
216         case 0:
217             return;
218         default:
219             System.out.println("Invalid option.");
220     }
221 }
222

```

```
222
223 public static void cartMenu(Cart cart, Scanner scanner) {
224     int choice = -1;
225     while (choice != 0) {
226         System.out.println("Options: ");
227         System.out.println("-----");
228         System.out.println("1. Filter medias in cart");
229         System.out.println("2. Sort medias in cart");
230         System.out.println("3. Remove media from cart");
231         System.out.println("4. Play a media");
232         System.out.println("5. Place order");
233         System.out.println("0. Back");
234         System.out.println("-----");
235         System.out.print("Please choose a number: 0-1-2-3-4-5: ");
236         choice = scanner.nextInt();
237         scanner.nextLine();
238         switch (choice) {
239             case 1:
240                 filterCart(cart, scanner);
241                 break;
242             case 2:
243                 sortCart(cart, scanner);
244                 break;
245             case 3:
246                 removeMediaFromCart(cart, scanner);
247                 break;
248             case 4:
249                 playMediaFromCart(cart, scanner);
250                 break;
251             case 5:
252                 placeOrder(cart);
253                 break;
254             case 0:
255                 return;
256             default:
257                 System.out.println("Invalid option.");
258         }
259     }
260 }
261
```

```
261
262     public static void filterCart(Cart cart, Scanner scanner) {
263         System.out.println("Filter by: ");
264         System.out.println("1. ID");
265         System.out.println("2. Title");
266         int filterChoice = scanner.nextInt();
267         scanner.nextLine();
268         switch (filterChoice) {
269             case 1:
270                 System.out.print("Enter media ID: ");
271                 int id = scanner.nextInt();
272                 Media media = cart.getItemsOrdered().stream()
273                     .filter(m -> m.getId() == id)
274                     .findFirst()
275                     .orElse(null);
276                 if (media != null) {
277                     media.displayInfo();
278                 } else {
279                     System.out.println("Media not found.");
280                 }
281                 break;
282             case 2:
283                 System.out.print("Enter media title: ");
284                 String title = scanner.nextLine();
285                 cart.getItemsOrdered().stream()
286                     .filter(m -> m.getTitle().contains(title))
287                     .forEach(Media::displayInfo);
288                 break;
289             default:
290                 System.out.println("Invalid choice.");
291         }
292     }
293 }
```

```

293
294 public static void sortCart(Cart cart, Scanner scanner) {
295     System.out.println("Sort by: ");
296     System.out.println("1. Title");
297     System.out.println("2. Cost");
298     int sortChoice = scanner.nextInt();
299     scanner.nextLine();
300     switch (sortChoice) {
301         case 1:
302             Collections.sort(cart.getItemsOrdered(), new CompareByTitleThenCost());
303             break;
304         case 2:
305             Collections.sort(cart.getItemsOrdered(), new CompareByCostThenTitle());
306             break;
307         default:
308             System.out.println("Invalid choice.");
309             return;
310     }
311     cart.displayCart();
312 }
313
314 public static void removeMediaFromCart(Cart cart, Scanner scanner) {
315     System.out.print("Enter media title to remove: ");
316     String title = scanner.nextLine();
317     Media mediaToRemove = cart.getItemsOrdered().stream()
318         .filter(m -> m.getTitle().equalsIgnoreCase(title))
319         .findFirst()
320         .orElse(null);
321     if (mediaToRemove != null) {
322         cart.removeMedia(mediaToRemove);
323     } else {
324         System.out.println("Media not found.");
325     }
326 }
327
328 public static void playMediaFromCart(Cart cart, Scanner scanner) {
329     System.out.print("Enter media title to play: ");
330     String title = scanner.nextLine();
331     Media mediaToPlay = cart.getItemsOrdered().stream()
332         .filter(m -> m.getTitle().equalsIgnoreCase(title))
333         .findFirst()
334         .orElse(null);
335     if (mediaToPlay != null && mediaToPlay instanceof Playable) {
336         ((Playable) mediaToPlay).play();
337     } else {
338         System.out.println("Media not found or cannot be played.");
339     }
340 }

```

```

340     }
341
342     public static void placeOrder(Cart cart) {
343         System.out.println("Order placed! Total Cost: $" + cart.totalCost());
344         cart.getItemsOrdered().clear();
345         System.out.println("Cart has been cleared.");
346     }
347 }

```

## 12. Question

1. Các lớp nào là tập hợp của các lớp khác? Kiểm tra tất cả các constructor của các lớp để xem chúng có khởi tạo các phần của chúng không?

Aggregate classes: Trong một hệ thống lập trình hướng đối tượng, một lớp được gọi là aggregate khi nó chứa các đối tượng của các lớp khác dưới dạng các trường (fields) hoặc thuộc tính (properties). Một lớp sẽ trở thành aggregate của các lớp khác nếu nó chứa các đối tượng của lớp khác trong các trường của mình.

Các lớp trong AimsProject có thể là aggregates:

Cart: Nếu lớp Cart chứa các đối tượng của lớp Media (hoặc các lớp con của Media như Book, DigitalVideoDisc, CompactDisc), thì Cart là một aggregate của các lớp Media hoặc các lớp con của nó.

Order: Nếu lớp Order chứa các đối tượng của lớp Cart hoặc Media, nó cũng sẽ là một aggregate.

CompactDisc: Lớp CompactDisc có thể là aggregate nếu nó chứa các đối tượng Track.

Kiểm tra các constructor:

Khi kiểm tra các constructor của lớp, bạn cần đảm bảo rằng chúng khởi tạo các trường (fields) là các đối tượng của các lớp khác. Nếu constructor khởi tạo các trường như vậy, thì lớp đó là một aggregate.

Ví dụ, constructor của lớp Cart có thể nhận danh sách các Media và lưu trữ chúng, điều này có thể được coi là việc khởi tạo các "phần tử" của lớp Cart.

## 2. Chuyện gì sẽ xảy ra nếu đối tượng truyền vào không phải là một thể hiện của lớp Media khi ghi đè phương thức equals() trong lớp Media?

Khi bạn ghi đè phương thức equals() trong lớp Media, nếu đối tượng được truyền vào không phải là một instance của lớp Media, bạn cần phải xử lý tình huống này một cách hợp lý. Cụ thể, trong phương thức equals(), bạn sẽ kiểm tra xem đối tượng truyền vào có phải là một instance của lớp Media hay không. Nếu không, phương thức sẽ trả về false, vì một đối tượng khác kiểu sẽ không thể coi là "bằng" với đối tượng Media hiện tại.

Giải thích chi tiết:

Khi bạn cài đặt phương thức equals() trong Java, bạn cần đảm bảo rằng nó so sánh các đối tượng của cùng một lớp hoặc các lớp con của nó. Nếu đối tượng truyền vào không phải là một instance của lớp bạn đang cài đặt (trong trường hợp này là Media), phương thức equals() sẽ trả về false, vì rõ ràng là hai đối tượng khác kiểu sẽ không thể bằng nhau.

Dưới đây là ví dụ về cách cài đặt phương thức equals() trong lớp Media và lớp Track để đáp ứng yêu cầu của bài toán.

Cài đặt phương thức equals() trong lớp Media:

@Override

```
public boolean equals(Object obj) {  
    // Nếu đối tượng truyền vào là null hoặc không phải là một instance của Media  
    if (obj == null || !(obj instanceof Media)) {  
        return false;  
    }  
  
    // Ép kiểu đối tượng obj thành Media để so sánh  
    Media otherMedia = (Media) obj;
```

```
// So sánh các thuộc tính (title) của đối tượng Media
return this.title != null && this.title.equals(otherMedia.title);
}
```

Giải thích:

Kiểm tra đối tượng null và kiểu đối tượng: Đầu tiên, chúng ta kiểm tra nếu đối tượng truyền vào là null hoặc không phải là một instance của lớp Media. Trong trường hợp này, phương thức trả về false.

Ép kiểu đối tượng: Sau khi chắc chắn rằng đối tượng là Media, chúng ta ép kiểu obj thành kiểu Media để có thể truy cập các thuộc tính của đối tượng đó.

So sánh thuộc tính: Cuối cùng, chúng ta so sánh thuộc tính title của đối tượng hiện tại với đối tượng otherMedia. Nếu chúng giống nhau, phương thức sẽ trả về true, nghĩa là hai đối tượng Media này là giống nhau.

Cài đặt phương thức equals() trong lớp Track:

@Override

```
public boolean equals(Object obj) {
    // Nếu đối tượng truyền vào là null hoặc không phải là một instance của Track
    if (obj == null || !(obj instanceof Track)) {
        return false;
    }

    // Ép kiểu đối tượng obj thành Track để so sánh
    Track otherTrack = (Track) obj;

    // So sánh title và length của các track
    return this.title != null && this.title.equals(otherTrack.title)

```

```
&& this.length == otherTrack.length;  
}
```

Giải thích:

Kiểm tra đối tượng null và kiểu đối tượng: Giống như trong lớp Media, ta kiểm tra nếu đối tượng truyền vào là null hoặc không phải là một instance của Track. Nếu là vậy, trả về false.

Ép kiểu đối tượng: Nếu đối tượng là một instance của lớp Track, ta ép kiểu đối tượng thành Track để so sánh các thuộc tính.

So sánh thuộc tính: Sau đó, chúng ta so sánh cả title và length của track để xác định chúng có giống nhau hay không.

### 3. Lớp nào nên triển khai interface Comparable?

Lớp Media hoặc các lớp con của nó (ví dụ: DigitalVideoDisc, Book, CompactDisc,...) nên thực hiện giao diện Comparable. Vì chúng đều có các thuộc tính chung như title, cost,... và cần có khả năng sắp xếp, do đó, một trong các lớp này sẽ cần cài đặt compareTo() để xác định cách so sánh các đối tượng Media.

Nếu bạn muốn mỗi loại Media có thể tự sắp xếp, bạn sẽ cài đặt compareTo() trong lớp Media hoặc trong từng lớp con cụ thể (ví dụ: DigitalVideoDisc sẽ cài đặt compareTo() riêng của nó).

Nếu bạn sử dụng Comparable cho lớp Media, bạn có thể cài đặt một quy tắc sắp xếp mặc định, nhưng sẽ bị hạn chế về khả năng sắp xếp theo nhiều cách khác nhau (ví dụ: theo title, cost,...).

### 4. Trong những lớp đó, bạn nên triển khai phương thức compareTo() như thế nào để phản ánh thứ tự mà chúng ta muốn?

Phương thức compareTo() phải trả về một số nguyên cho biết thứ tự của đối tượng hiện tại so với một đối tượng khác:

Giả sử bạn sử dụng Comparable, phương thức compareTo() sẽ cần được cài đặt như sau:

Sắp xếp theo title, rồi cost: Bạn có thể so sánh title trước, và nếu chúng giống nhau, thì so sánh theo cost:

@Override

```
public int compareTo(Media other) {  
    int titleComparison = this.title.compareTo(other.title);  
    if (titleComparison != 0) {  
        return titleComparison;  
    } else {  
        return Float.compare(other.cost, this.cost); // Sắp xếp theo cost giảm dần  
    }  
}
```

Sắp xếp theo cost, rồi title: Bạn có thể so sánh cost trước, và nếu chúng giống nhau, thì so sánh theo title:

@Override

```
public int compareTo(Media other) {  
    int costComparison = Float.compare(other.cost, this.cost); // Sắp xếp theo cost giảm dần  
    if (costComparison != 0) {  
        return costComparison;  
    } else {  
        return this.title.compareTo(other.title); // Sắp xếp theo title  
    }  
}
```

Trong hai ví dụ trên, chúng ta thực hiện sắp xếp theo 2 thuộc tính, nhưng chỉ có một quy tắc sắp xếp có thể được sử dụng trong `compareTo()`. Do đó, mỗi lớp chỉ có thể có một quy tắc sắp xếp mặc định.

5. Chúng ta có thể có hai quy tắc sắp xếp của mục (theo tiêu đề rồi đến chi phí và theo chi phí rồi đến tiêu đề) nếu chúng ta sử dụng cách tiếp cận interface `Comparable` không?

Không, nếu bạn sử dụng giao diện `Comparable`, mỗi lớp chỉ có thể có một quy tắc sắp xếp duy nhất. Điều này có nghĩa là bạn chỉ có thể quyết định sắp xếp theo title rồi cost hoặc theo cost rồi title trong `compareTo()`. Nếu bạn muốn có nhiều cách sắp xếp, bạn phải sử dụng `Comparator`, vì `Comparator` cho phép bạn định nghĩa nhiều cách sắp xếp khác nhau mà không cần phải thay đổi mã của lớp `Media`.

6. Giả sử các DVD có quy tắc sắp xếp khác với các loại phương tiện khác, đó là theo tiêu đề, sau đó là độ dài giảm dần, rồi đến chi phí. Bạn sẽ sửa đổi mã của mình như thế nào để cho phép điều này?

Nếu bạn muốn có quy tắc sắp xếp riêng cho `DigitalVideoDisc` (DVD), bạn cần phải cài đặt `Comparable` cho `DigitalVideoDisc` và thực hiện phương thức `compareTo()` khác với `Media`.

Ví dụ, trong lớp `DigitalVideoDisc`, bạn có thể cài đặt `compareTo()` như sau:

@Override

```
public int compareTo(Media other) {  
    if (other instanceof DigitalVideoDisc) {  
        DigitalVideoDisc otherDVD = (DigitalVideoDisc) other;  
  
        // Sắp xếp theo title  
        int titleComparison = this.title.compareTo(otherDVD.title);
```

```
if (titleComparison != 0) {
    return titleComparison;
}

// Sắp xếp theo length giảm dần
int lengthComparison = Integer.compare(otherDVD.length, this.length); // decreasing order
if (lengthComparison != 0) {
    return lengthComparison;
}

// Sắp xếp theo cost
return Float.compare(otherDVD.cost, this.cost); // decreasing order
}

return super.compareTo(other); // Nếu không phải DVD, sử dụng phương thức so sánh của
Media
}
```